

ADR 4 – Hoe richten we RabbitMQ in?

Datum 11-05-2026

Groep C8

Wat is het vraagstuk?

De communicatiemodule moet betrouwbaar berichten kunnen versturen naar externe messaging providers. Daarbij moet de module kunnen omgaan met tijdelijke storingen, time-outs en downtime van providers zonder dat berichten verloren gaan.

De opdracht vereist expliciet:

- Retry-mechanismen bij fouten
- Dead letter queue ondersteuning
- Betrouwbare aflevering van berichten
- Schaalbaarheid naar meerdere organisaties
- Inzicht in foutmeldingen via monitoring

Daarom moesten we bepalen hoe we RabbitMQ precies inrichten:

- Welke queues hebben we nodig?
- Hoe verwerken we retries?
- Hoe voorkomen we dat berichten verloren gaan?
- Hoe houden we het systeem overzichtelijk en schaalbaar?

Waarom RabbitMQ?

In ADR 2 is al besloten dat RabbitMQ de beste keuze is als berichtenwachtrij. De belangrijkste redenen waren:

- Ondersteuning voor retries en dead letter queues
- Eenvoudiger op te zetten dan Kafka
- Goed geschikt voor het verwachte volume
- Goede integratie met Spring Boot
- Betrouwbare aflevering van berichten

RabbitMQ past goed bij ons scenario omdat notificaties niet real-time op milliseconde-niveau hoeven te zijn, maar wel betrouwbaar moeten aankomen.

Mogelijke inrichting

We hebben drie mogelijke ontwerpen besproken.

Optie A: Eén centrale queue voor alles

Alle berichten van alle organisaties en providers komen in één queue terecht.

Voordelen:

- Eenvoudig op te zetten
- Minder configuratie nodig
- Minder queues om te beheren

Nadelen:

- Moeilijker te debuggen
- Problemen bij één provider beïnvloeden alle berichten
- Minder schaalbaar
- Geen duidelijke scheiding tussen providers

Deze optie vonden we uiteindelijk te beperkt.

Optie B: Eén queue per provider (onze keuze)

Elke messaging provider krijgt een eigen queue.

Voorbeeld:

- queue.swiftsend
- queue.legacylink
- queue.asyncflow
- queue.securepost

Daarnaast krijgt iedere provider:

- Een retry queue
- Een dead letter queue (DLQ)

Voordelen:

- Problemen bij één provider beïnvloeden de rest niet
- Overzichtelijker monitoring
- Eenvoudiger schalen per provider
- Makkelijk retries apart configureren
- Duidelijke logging en foutanalyse

Nadelen:

- Meer configuratie
- Meer queues om te beheren

Deze nadelen wegen niet op tegen de voordelen.

Optie C: Eén queue per organisatie

Elke organisatie krijgt een aparte queue.

Voordelen:

- Volledige scheiding per ziekenhuis
- Organisaties beïnvloeden elkaar niet

Nadelen:

- Grote hoeveelheid queues bij veel klanten
- Complex beheer
- Meer infrastructuurkosten
- Lastiger monitoren

Omdat onze providers belangrijker zijn voor de verwerking dan de organisaties, hebben we deze optie niet gekozen.

Beslissing

We kiezen voor optie B: één queue per messaging provider, aangevuld met retry queues en dead letter queues.

Inrichting van RabbitMQ

Hoofdqueues

Elke provider krijgt een eigen hoofdqueue:

Queue	Doel
queue.swiftsend	Berichten voor SwiftSend
queue.legacylink	Berichten voor LegacyLink
queue.asyncflow	Berichten voor AsyncFlow
queue.securepost	Berichten voor SecurePost

De scheduler en businesslogica plaatsen berichten in de juiste queue afhankelijk van de provider die een organisatie gebruikt.

Retry queues

Bij tijdelijke fouten gaat een bericht eerst naar een retry queue.

Voorbeelden:

- `retry.swiftsend`
- `retry.legacylink`

De retry queue gebruikt een time-to-live (TTL). Na afloop daarvan stuurt RabbitMQ het bericht automatisch terug naar de oorspronkelijke queue.

Dit voorkomt dat de applicatie zelf ingewikkelde wachttijden moet beheren.

Dead letter queues (DLQ)

Als een bericht na meerdere retries nog steeds mislukt, gaat het naar een dead letter queue.

Voorbeelden:

- `dlq.swiftsend`
- `dlq.legacylink`

Berichten in de DLQ worden niet automatisch opnieuw geprobeerd.

Beheerders kunnen via het dashboard:

- De foutmelding bekijken
- Handmatig opnieuw proberen
- Het bericht markeren als definitief mislukt

Retry-strategie

We gebruiken exponential backoff om providers niet te overbelasten.

Voorbeeld:

Poging	Wachttijd
1	30 seconden
2	2 minuten
3	10 minuten
4	Naar DLQ

Hierdoor krijgt een tijdelijke storing tijd om te herstellen zonder dat RabbitMQ direct duizenden retries uitvoert.

Betrouwbaarheid

Om verlies van berichten te voorkomen gebruiken we:

- Durable queues
- Persistent messages
- Acknowledgements vanuit consumers
- Dead letter exchanges

Daardoor blijven berichten bestaan als RabbitMQ opnieuw opstart.

Pas nadat een provider succesvol heeft bevestigd dat een bericht is verwerkt, wordt het bericht uit de queue verwijderd.

Monitoring

RabbitMQ wordt gekoppeld aan OpenTelemetry, Prometheus en Grafana.

We monitoren onder andere:

- Grootte van queues
- Aantal retries
- Aantal berichten in DLQ
- Verwerkingstijd per provider
- Foutpercentages

Hierdoor kunnen beheerders snel zien wanneer een provider problemen heeft.

Gevolgen

Positieve gevolgen

- Betrouwbare aflevering van berichten
- Providers zijn van elkaar gescheiden
- Tijdelijke storingen worden automatisch opgevangen
- Goede schaalbaarheid
- Duidelijke monitoring en foutanalyse

Negatieve gevolgen

- Meer RabbitMQ-configuratie
- Meer infrastructuurcomponenten
- Complexere monitoring dan zonder wachtrij

Deze nadelen vinden we acceptabel omdat betrouwbaarheid belangrijker is dan eenvoud.

Relatie met andere ADR's

ADR	Relatie
ADR 1	RabbitMQ draait in de zelfstandige SaaS-module
ADR 2	RabbitMQ is gekozen als berichtenwachtrij
ADR 3	Polling levert berichten aan die via RabbitMQ verwerkt worden

Voorbeeld van de berichtstroom

1. De polling service haalt een afspraak op via de FHIR API.
2. De businesslogica bepaalt welke provider gebruikt moet worden.
3. Het bericht wordt in de juiste provider queue geplaatst.
4. Een consumer verstuurt het bericht naar de provider.
5. Bij succes wordt het bericht bevestigd (ack).
6. Bij een tijdelijke fout gaat het bericht naar de retry queue.
7. Na meerdere mislukte pogingen gaat het bericht naar de DLQ.
8. Grafana toont de status aan beheerders.

Conclusie

Met deze RabbitMQ-inrichting voldoet de communicatiemodule aan de eisen voor betrouwbaarheid, retries en monitoring. Door queues per provider te gebruiken houden we het systeem schaalbaar, overzichtelijk en beter bestand tegen storingen bij externe messaging providers.